

Research Software Engineering in Python for Reproducible Computational Science

Sylwester Arabas

AGH University of Krakow

2026-03-19



AGH



previously on...

- ▶ what (or who) is **RSE**?
- ▶ Python as a glue language
- ▶ unit-aware coding/plotting and dimensional analysis (e.g., **Pint**)



AGH



previously on...

- ▶ what (or who) is **RSE**?
- ▶ Python as a glue language
- ▶ unit-aware coding/plotting and dimensional analysis (e.g., **Pint**)
- ▶ **reproducibility** in computational science (and beyond)
- ▶ automation & reuse (DRY)
- ▶ **SciPy** (but what about SciPy+Pint?!)



AGH



plan for today

- SciPy + Pint: how to couple?
- JIT-compilation as an alternative to pre-compiled libs (code along!)
- MPI & Numba-MPI



AGH



plan for today

- **SciPy + Pint: how to couple?**
- JIT-compilation as an alternative to pre-compiled libs (code along!)
- MPI & Numba-MPI



AGH



equilibrium-vapour pressure (Teten's formula)

```
import pint
import numpy as np
from scipy import constants

si = pint.UnitRegistry()
```



AGH



equilibrium-vapour pressure (Teten's formula)

```
import pint
import numpy as np
from scipy import constants

si = pint.UnitRegistry()

def equilibrium_vapour_pressure(T):
    """ Teten's formula (https://en.wikipedia.org/wiki/Tetens\_equation) """
    TC = T - constants.zero_Celsius * si.K
    return .61078 * si.kPa * np.exp(17.27 * TC / (237.3 * si.K + TC))
```



AGH



equilibrium-vapour pressure (Teten's formula)

```
import pint
import numpy as np
from scipy import constants

si = pint.UnitRegistry()

def equilibrium_vapour_pressure(T):
    """ Teten's formula (https://en.wikipedia.org/wiki/Tetens\_equation) """
    TC = T - constants.zero_Celsius * si.K
    return .61078 * si.kPa * np.exp(17.27 * TC / (237.3 * si.K + TC))

print(f"{equilibrium_vapour_pressure(T=300 * si.K)=:.2g~}")

equilibrium_vapour_pressure(T=300 * si.K)=3.5 kPa
```



AGH



SciPy.optimize.root_scalar

scipy.optimize.

root_scalar

```
root_scalar(f, args=(), method=None, bracket=None, fprime=None, fprime2=None, x0=None,
x1=None, xtol=None, rtol=None, maxiter=None, options=None)
```

Find a root of a scalar function.

Parameters:

f : *callable*

A function to find a root of.

Suppose the callable has signature `f0(x, *my_args, **my_kwargs)`, where `my_args` and `my_kwargs` are required positional and keyword arguments. Rather than passing `f0` as the callable, wrap it to accept only `x`; e.g., pass `fun=lambda x: f0(x, *my_args, **my_kwargs)` as the callable, where `my_args` (tuple) and `my_kwargs` (dict) have been gathered before invoking this function.

args : *tuple, optional*

Extra arguments passed to the objective function and its derivative(s).

method : *str, optional*

Type of solver. Should be one of

- 'bisect'
-
-
-

bracket : *A sequence of 2 floats, optional*

An interval bracketing a root. `f(x, *args)` must have different signs at the two endpoints.

xtol : *float, optional*

Tolerance (absolute) for termination.



AGH



using SciPy to numerically determine f^{-1}

```
from scipy.optimize import root_scalar
```

```
problem = lambda T: equilibrium_vapour_pressure(T) - p_atm
```



AGH



using SciPy to numerically determine f^{-1}

```
from scipy.optimize import root_scalar

problem = lambda T: equilibrium_vapour_pressure(T) - p_atm

try:
    res = root_scalar(f=problem, bracket=[300 * si.K, 400 * si.K])
except Exception as e:
    print(str(e))
```

Unable to parse arguments



AGH



10/33

what's wrong?!

ChatGPT ▾

Log in

Sign up for free

Hi, I'm trying to use `scipy.optimize.root_scalar` the following way (where `si` is an instance of `pint.UnitRegistry`):

```
res = root_scalar(lambda T:
equilibrium_vapour_pressure(T) - p_atm,
    bracket=[300 * si.K, 400 * si.K], method='bisect',
    xtol=0.1 * si.K)
```

but it raises an exception with "Unable to parse arguments" message, how to fix it?

Ah! The issue here is that `scipy.optimize.root_scalar` **does not natively understand Pint quantities with units**. It expects **plain Python floats** for `x0`, `x1`, `bracket`, and `xtol`. When you pass `300 * si.K` or `0.1 * si.K`, it raises the "Unable to parse arguments" error because it tries to convert a Pint `Quantity` to a float internally and fails.

Here's how you can fix it:

1. Strip units before passing to `root_scalar`

let's solve it properly!



AGH



11/33

applying Python's “duck typing” for IoC

```
def equilibrium_vapour_pressure(si, T):  
    """ Teten's formula (https://en.wikipedia.org/wiki/Tetens\_equation) """  
    TC = T - constants.zero_Celsius * si.K  
    return .61078 * si.kPa * np.exp(17.27 * TC / (237.3 * si.K + TC))
```



AGH



applying Python's "duck typing" for IoC

```
def equilibrium_vapour_pressure(si, T):  
    """ Teten's formula (https://en.wikipedia.org/wiki/Tetens\_equation) """  
    TC = T - constants.zero_Celsius * si.K  
    return .61078 * si.kPa * np.exp(17.27 * TC / (237.3 * si.K + TC))
```

```
si = pint.UnitRegistry()  
print(f"{equilibrium_vapour_pressure(si, T=300 * si.K)=:.2g~}")
```

```
equilibrium_vapour_pressure(si, T=300 * si.K)=3.5 kPa
```



AGH



13/33

applying Python's "duck typing" for IoC

```
from collections import namedtuple
```

```
fsi = namedtuple("FakeUnitRegistry", ("K", "kPa"))(K=1, kPa=1000)  
print(type(fsi))
```

```
<class '__main__.FakeUnitRegistry'>
```



AGH



applying Python's "duck typing" for IoC

```
from collections import namedtuple
```

```
fsi = namedtuple("FakeUnitRegistry", ("K", "kPa"))(K=1, kPa=1000)  
print(type(fsi))
```

```
<class '__main__.FakeUnitRegistry'>
```

```
print(f"{equilibrium_vapour_pressure(fsi, T=300 * fsi.K)=:.2g}")
```

```
equilibrium_vapour_pressure(fsi, T=300 * fsi.K)=3.5e+03
```



AGH



works with SciPy! (code can still be unit-aware tested)

```
problem = lambda T, p_atm, si: equilibrium_vapour_pressure(si, T) - p_atm
```



AGH



works with SciPy! (code can still be unit-aware tested)

```
problem = lambda T, p_atm, si: equilibrium_vapour_pressure(si, T) - p_atm  
res = root_scalar(f=problem, bracket=[300 * fsi.K, 400 * fsi.K],  
                  args=(50 * fsi.kPa, fsi,))
```



AGH



works with SciPy! (code can still be unit-aware tested)

```
problem = lambda T, p_atm, si: equilibrium_vapour_pressure(si, T) - p_atm  
res = root_scalar(f=problem, bracket=[300 * fsi.K, 400 * fsi.K],  
                  args=(50 * fsi.kPa, fsi,))  
  
print(res)
```

```
    converged: True  
      flag: converged  
function_calls: 11  
iterations: 10  
    root: 354.4029891789775  
    method: brentq
```



AGH



18/33

works with SciPy! (code can still be unit-aware tested)

```
problem = lambda T, p_atm, si: equilibrium_vapour_pressure(si, T) - p_atm  
res = root_scalar(f=problem, bracket=[300 * fsi.K, 400 * fsi.K],  
                  args=(50 * fsi.kPa, fsi,))
```

```
print(res)
```

```
    converged: True  
      flag: converged  
function_calls: 11  
  iterations: 10  
      root: 354.4029891789775  
    method: brentq
```

```
assert problem(300 * si.K, 1000 * si.hPa, si).check("[pressure]")
```



AGH



plan for today

- SciPy + Pint: how to couple?
- **JIT-compilation as an alternative to pre-compiled libs (code along!)**
- MPI & Numba-MPI



AGH



toy problem: Monte-Carlo π estimation in Python

```
from random import random as u01
pi = lambda n: 4 * sum(abs(u01() + 1j * u01()) < 1 for _ in range(n)) / n
```

(note: use of generator $\rightsquigarrow$ reduced memory footprint vs. list comprehension)



AGH



toy problem: Monte-Carlo π estimation in Python

```
from random import random as u01
pi = lambda n: 4 * sum(abs(u01() + 1j * u01()) < 1 for _ in range(n)) / n
```

(note: use of generator $\rightsquigarrow$ reduced memory footprint vs. list comprehension)

```
%%time
f"{pi(2**23):.4g}"
```

CPU times: user 1.73 s, sys: 950 μ s, total: 1.73 s

Wall time: 1.76 s

'3.142'



AGH





Numba makes Python code fast

Numba is an open source JIT compiler that translates a subset of Python and NumPy code into fast machine code.

[Learn More](#)[Try Numba »](#)

Accelerate Python Functions

Numba translates Python functions to optimized machine code at runtime using the industry-standard [LLVM](#) compiler library. Numba-compiled numerical algorithms in Python can approach the speeds of C or FORTRAN.

You don't need to replace the Python interpreter, run a separate compilation step, or even have a C/C++ compiler installed. Just apply one of the Numba decorators to your Python function, and Numba does the rest.

[Learn More »](#)[Try Now »](#)

```
from numba import njit
import random

@njit
def monte_carlo_pi(nsamples):
    acc = 0
    for i in range(nsamples):
        x = random.random()
        y = random.random()
        if (x ** 2 + y ** 2) < 1.0:
            acc += 1
    return 4.0 * acc / nsamples
```

Built for Scientific Computing

Numba is designed to be used with NumPy arrays and functions. Numba generates specialized code for different array data types and layouts to optimize performance. Special decorators can create [universal functions](#) that broadcast over NumPy arrays just like NumPy functions do.

```
@njit(parallel=True)
def logistic_regression(Y, X, w, iteratic
    for i in range(iterations):
        w -= np.dot(((1.0 /
                    (1.0 + np.exp(-Y * np.dot(
                        - 1.0) * Y), X)
                    )
```

Numba also works great with Jupyter notebooks for interactive computing, and with distributed execution frameworks, like Dask and Spark.

[Learn More »](#)[Try Now »](#)

Parallelize Your Algorithms

Numba offers a range of options for parallelizing your code for CPUs and GPUs, often with only minor code changes.

Simplified Threading

```
@njit(parallel=True)
def simulator(out):
    # iterate loop in parallel
    for i in prange(out.shape[0]):
        out[i] = run_sim()
```

Numba can automatically execute NumPy array expressions on multiple CPU cores and makes it easy to write parallel loops.

[Learn More »](#)[Try Now »](#)

SIMD Vectorization

```
LBB@_8:
vmovups {%rax,%rdx,4), %ymm0
vmovups {%rcx,%rdx,4), %ymm1
vsubps %ymm1, %ymm0, %ymm2
vaddps %ymm2, %ymm2, %ymm2
```

Numba can automatically translate some loops into vector instructions for 2-4x speed improvements. Numba adapts to your CPU capabilities, whether your CPU supports SSE, AVX, or AVX-512.

[Learn More »](#)[Try Now »](#)

GPU Acceleration



With support for NVIDIA CUDA, Numba lets you write parallel GPU algorithms entirely from Python.

[Numba CUDA »](#)

toy problem: let's speed it up with Numba JIT compilation

```
import numba

try:
    numba.jit(pi)(2**23)
except Exception as e:
    print(e.msg)
```

The use of yield in a closure is unsupported.



AGH



24/33

toy problem: let's speed it up with Numba JIT compilation

```
@numba.jit
def pij(n):
    c = 0
    for _ in range(n):
        c += abs(u01() + 1j * u01()) < 1
    return 4 * c / n
```



AGH



toy problem: let's speed it up with Numba JIT compilation

```
@numba.jit
def pij(n):
    c = 0
    for _ in range(n):
        c += abs(u01() + 1j * u01()) < 1
    return 4 * c / n
```

```
%%time
f"{pij(2**23):.4g}"
```

CPU times: user 492 ms, sys: 995 μ s, total: 493 ms

Wall time: 500 ms

'3.141'

note: this includes the time of compilation!



AGH



plan for today

- SciPy + Pint: how to couple?
- JIT-compilation as an alternative to pre-compiled libs (code along!)
- **MPI & Numba-MPI**



AGH



Message Passing Interface

The **Message Passing Interface** (MPI) is a portable message-passing standard designed to function on parallel computing architectures.^[1] The MPI standard defines the syntax and semantics of library routines that are useful to a wide range of users writing portable message-passing programs in C, C++, and Fortran. There are several open-source MPI implementations, which fostered the development of a parallel software industry, and encouraged development of portable and scalable large-scale parallel applications.

MPI @ Cyfronet (or any other supercomputing centre)



AGH



Contents



numba-mpi

Submodules

api

common

utils

API Documentation

SUCCESS

built with 

► View Source

numba_mpi



numba-mpi



Overview

numba-mpi provides Python wrappers to the C MPI API callable from within [Numba JIT-compiled code](#) (@jit mode). For an outline of the project, rationale, architecture, and features, refer to: [numba-mpi paper in SoftwareX \(open access\)](#) (please cite if numba-mpi is used in your research).

Support is provided for a subset of MPI routines covering: `size / rank`, `send / recv`, `allreduce`, `reduce`, `bcast`, `scatter / gather` & `allgather`, `barrier`, `wtime` and basic asynchronous communication with `isend / irecv` (only for contiguous arrays); for request handling including `wait / waitall / waitany` and `test / testall / testany`.

The API uses NumPy and supports both numeric and character datatypes (e.g., `broadcast`). Auto-generated docstring-based API docs are published on the web: <https://numba-mpi.github.io/numba-mpi>

Packages can be obtained from [PyPI](#), [Conda Forge](#), [Arch Linux](#) or by invoking `pip install`



AGH



30/33



ScienceDirect®

SoftwareX

Volume 28, December 2024, 101897

Original software publication

Numba-MPI v1.0: Enabling MPI communication within Numba/LLVM JIT-compiled Python code

Kacper Derlatka ^{a 1}, Maciej Manna ^{a 2}, Oleksii Bulenok ^{a 3}, David Zwicker ^b, Sylwester Arabas ^c  

^a Faculty of Mathematics and Computer Science, Jagiellonian University in Kraków, Poland

^b Max Planck Institute for Dynamics and Self-Organization, Göttingen, Germany

^c Faculty of Physics and Applied Computer Science, AGH University of Krakow, Poland

Received 10 July 2024, Revised 4 September 2024, Accepted 9 September 2024, Available online 4 October 2024,
Version of Record 4 October 2024.



AGH



31/33

Python $\rightsquigarrow$ expressive glue language
(and good gluing is a craft)

Numba-MPI $\rightsquigarrow$ contributors welcome!

Thank you for your attention!

slides and other course resources at
<https://github.com/open-atmos-krk/rse-python-course>



AGH

